```
struct X
{
    int i;
    std::string s;
};

std::atomic<X*> p;
std::atomic<int> a;

void create_x()
{
    X* x=new X;
    x->i=42;
    x->s="hello";
    a.store(99,std::memory_order_relaxed);
    p.store(x,std::memory_order_release);
}


void use_x()
{
    X* x;
    while(!(x=p.load(std::memory_order_consume)))
        std::this_thread::sleep(std::chrono::microseconds(1));
    assert(x->i==42);
    assert(x->s=="hello");
    assert(a.load(std::memory_order_relaxed)==99);
}

int main()
{
    std::thread t1(create_x);
    std::thread t2(use_x);
    t1.join();
    t2.join();
}
```

Even though the store to `a` ❶ is sequenced before the store to `p` ❷, and the store to `p` is tagged `memory_order_release`, the load of `p` ❸ is tagged `memory_order_consume`. This means that the store to `p` only happens-before those expressions that are dependent on the value loaded from `p`. This means that the asserts on the data members of the `X` structure ❹, ❺ are guaranteed not to fire, because the load of `p` carries a dependency to those expressions through the variable `x`. On the other hand, the assert on the value of `a` ❻ may or may not fire; this operation isn't dependent on the value loaded from `p`, and so there's no guarantee on the value that's read. This is particularly apparent because it's tagged with `memory_order_relaxed`, as you'll see.

Sometimes, you don't want the overhead of carrying the dependency around. You want the compiler to be able to cache values in registers and reorder operations to optimize the code rather than fussing about the dependencies. In these scenarios, you can use `std::kill_dependency()` to explicitly break the dependency chain. `std::kill_dependency()` is a simple function template that copies the supplied argument to the return value but breaks the dependency chain in doing so. For example, if you have a global read-only array, and you use `std::memory_order_consume` when retrieving an